

LAB 7: Xây dựng Đồng hồ trực tuyến với useEffect

Để hiểu rõ cách useEffect hoạt động trong thực tế, chúng ta sẽ cùng xây dựng một ứng dụng nhỏ: **Đồng hồ trực tuyến (Live Clock)** có tính năng Bật/Tắt.

Bài thực hành này sẽ minh họa hoàn hảo cả 3 giai đoạn: **Mount** (Bắt đầu đếm giờ), **Update** (Cập nhật tiêu đề tab trình duyệt theo thời gian) và **Unmount** (Dọn dẹp bộ đếm giờ khi tắt đồng hồ để tránh rò rỉ bộ nhớ).

Dưới đây là các bước thực hiện mã nguồn bằng React Functional Component.

Bước 1: Khởi tạo Component chính (App) và tính năng Ẩn/Hiện

Đầu tiên, chúng ta tạo component App làm khung chứa. Điểm mấu chốt ở đây là biến state showClock. Việc thay đổi state này từ true sang false sẽ quyết định việc component đồng hồ được **Mount** (sinh ra) hay **Unmount** (hủy bỏ) khỏi DOM.

```
import React, { useState } from 'react';
import LiveClock from './LiveClock'; // Sẽ tạo ở Bước 2

export default function App() {
  const [showClock, setShowClock] = useState(true);

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial' }}>
      <h1>Thực hành vòng đời với useEffect</h1>

      {/* Nút bật/tắt để kích hoạt Mount và Unmount */}
      <button
        onClick={() => setShowClock(!showClock)}
        style={{ marginBottom: '20px', padding: '10px 20px', cursor: 'pointer' }}
      >
        {showClock ? "Ẩn Đồng Hồ (Unmount)" : "Hiện Đồng Hồ (Mount)}
      </button>

      {/* Render có điều kiện */}
      {showClock && <LiveClock />}
    </div>
  );
}
```

Bước 2: Xây dựng LiveClock với Giai đoạn Mount & Unmount

Bây giờ, tạo component LiveClock. Chúng ta sẽ dùng useEffect với **mảng phụ thuộc rỗng []** để tạo ra một bộ đếm setInterval ngay khi đồng hồ xuất hiện, và một hàm cleanup để xóa nó khi đồng hồ biến mất.

```
import React, { useState, useEffect } from 'react';

export default function LiveClock() {
  const [time, setTime] = useState(new Date());

  // Effect 1: Quản lý Timer (Tương đương componentDidMount + componentWillUnmount)
  useEffect(() => {
    console.log("[Mount]: Component đã xuất hiện trên DOM. Bắt đầu đếm giờ!");

    // Setup code: Chạy timer mỗi giây
    const timerId = setInterval(() => {
      setTime(new Date());
    }, 1000);

    // Cleanup function: Dọn dẹp trước khi component bị hủy
    return () => {
      console.log("[Unmount]: Component sắp bị hủy. Đã xóa timer!");
      clearInterval(timerId); // Quan trọng: Tránh rò rỉ bộ nhớ
    };
  }, []); // Mảng rỗng [] đảm bảo Effect này chỉ thiết lập 1 lần duy nhất

  return (
    <div style={{ padding: '20px', border: '2px solid #4CAF50', borderRadius: '8px' }}>
      <h2>Thời gian hiện tại:</h2>
      <h3>{time.toLocaleTimeString()}</h3>
    </div>
  );
}
```

Bước 3: Thêm logic Giai đoạn Update vào LiveClock

Tiếp theo, chúng ta muốn tiêu đề (title) của tab trình duyệt thay đổi mỗi khi thời gian nhảy số. Ta sẽ thêm một useEffect thứ hai vào LiveClock, sử dụng state time làm mảng phụ thuộc.

```
// Thêm đoạn code này vào bên trong component LiveClock (dưới Effect 1)

// Effect 2: Cập nhật tiêu đề trang (Tương đương componentDidUpdate)
useEffect(() => {
```

```
console.log("[Update]: Thời gian đã thay đổi, cập nhật tiêu đề tab.");

document.title = time.toLocaleTimeString();

// Khôi phục lại tiêu đề gốc khi đồng hồ bị ẩn đi
return () => {
  document.title = "Thực hành React";
};
}, [time])); // Phụ thuộc vào 'time': Chạy lại mỗi khi 'time' có giá trị mới
```

Phân tích kết quả chạy thực tế

Nếu mở màn hình Console trên trình duyệt (F12) và chạy ứng dụng này, sẽ quan sát thấy chuỗi sự kiện diễn ra đúng như vòng đời của React:

1. Khi bạn vừa load trang (Mặc định đồng hồ hiện):

- Giao diện hiển thị thời gian hiện tại.
- **Console:** [Mount]: Component đã xuất hiện trên DOM. Bắt đầu đếm giờ!
- **Console:** [Update]: Thời gian đã thay đổi, cập nhật tiêu đề tab.
- *Tab trình duyệt bắt đầu nhấp nháy thời gian thực.*

2. Mỗi 1 giây trôi qua (Đồng hồ đang chạy):

- setInterval gọi hàm setTime, làm state time thay đổi. Component re-render.
- Effect 1 (Timer) **không** chạy lại vì có mảng phụ thuộc rỗng [].
- Effect 2 (Title) chạy lại vì time thay đổi.
- **Console:** [Update]: Thời gian đã thay đổi... (xuất hiện liên tục mỗi giây).

3. Khi bạn click nút "Ẩn Đồng Hồ (Unmount)":

- Biến showClock trong App biến thành false. React quyết định gỡ LiveClock khỏi DOM.
- Trước khi xóa LiveClock, React gọi tất cả các hàm Cleanup.
- **Console:** [Unmount]: Component sắp bị hủy. Đã xóa timer!
- *Tiêu đề tab trình duyệt trở lại thành "Thực hành React". Bộ đếm ngưng hoạt động ở background.*

Bài học rút ra: Nếu ở **Bước 2**, bạn quên viết hàm `return () => clearInterval(timerId);`, thì mỗi lần Ấn rồi Hiện đồng hồ, ứng dụng sẽ sinh ra thêm một bộ đếm mới chạy ngầm song song với bộ đếm cũ, dẫn đến rò rỉ bộ nhớ, giật lag và lỗi giao diện nghiêm trọng.